Detection of crystals in medical images

Research and Innovation Project

MAJIC Petar

Supervisors

VENTURINI Gilles
SERRES Barthélémy
CHU Orléans

February 2026





Contents

1 Introduction and Project Overview **3**

1.1 Motivation . . . 3

1.2 Objectives . . . 3

1.3 Technologies and Tools . . . 3

2 Plan, Tasks and Deliverables **4**

3 Dataset review and normalisation **5**

3.1 Dataset overview . . . 5

3.2 Coding guide: crystal species codes . . . 5

3.3 Coding guide: abbreviations found in filenames . . . 6

3.4 Dataset issues identified during Phase 1 . . . 7

3.4.1 Folder organization and label reliability . . . 7

3.4.2 Morphological variability inside a species . . . 8

3.4.3 Filename annotations are not standardized . . . 8

3.4.4 Image Abbreviations . . . 8

3.5 Metadata CSV to normalize information . . . 8

3.5.1 How the CSV is built . . . 8

3.5.2 CSV fields . . . 9

3.5.3 Examples . . . 9

4 Dataset analysis **11**

4.1 Metadata CSV analysis . . . 11

4.1.1 Global split: non-mixture vs mixture . . . 11

4.1.2 Non-mixture images per crystal code . . . 11

4.1.3 Mixture images per crystal code . . . 12

5 Dataset Annotation **13**

5.1 Chosen annotation format . . . 13

5.2 Dataset structure . . . 13

5.2.1 Images . . . 13
5.2.2 Labels (segmentation annotations) . . . 14
5.2.3 data.yaml configuration . . . 14
5.3 Why YOLO segmentation format . . . 14

6 Model Fine Tuning (YOLO) 15
6.1 Dataset split: labeled vs unlabeled . . . 15
6.2 YOLO segmentation dataset format (training input) . . . 15
6.3 Dataset preparation and training protocol . . . 15
6.3.1 Cleaning and preparation before training . . . 16
6.3.2 Class filtering: keeping only classes with enough data . . . 16
6.3.3 Improved dataset variant: removing very small classes (drop4) . . . 17
6.3.4 Train/validation strategy: 5-fold stratified K-Fold . . . 17
6.3.5 Training configurations . . . 18
6.3.6 Evaluation metrics (segmentation masks) . . . 19
6.3.7 Baseline results (YOLO11n-seg, 19 classes) . . . 19
6.3.8 Improved results (YOLO11m-seg, drop4) . . . 20
6.3.9 Summary comparison (baseline vs improved) . . . 21
6.3.10 Validation batch visualization . . . 22

7 Conclusion 23

Chapter 1

Introduction and Project Overview

The crystals found in urine can be used to establish diagnoses for the detection of different diseases. In practice, they are analyzed by looking at images under a microscope. Using polarized light makes the crystal structures more visible and helps experts distinguish between different types of crystals.

In this project, we explore a deep learning approach for classifying urinary crystal images. The goal is to support medical experts and make their decisions more consistent and reliable. The project is carried out in collaboration with laboratories from Computer Science and the University Hospital Center (CHU) of Orléans.

1.1 Motivation

Manual diagnosis is time-consuming and subject to human variability. Deep learning can help standardize results and support experts by providing automatic predictions on microscopic images.

1.2 Objectives

The main objectives are:

- Collect and preprocess urinary crystal images.
- Explore annotation/segmentation tools to build a labeled dataset.
- Propose and implement a classification strategy using deep learning.
- Evaluate the results with medical experts and iterate.
- Deliver a fine-tuned model in Google Colab.

1.3 Technologies and Tools

- **Programming:** Python
- **Annotation:** CVAT
- **Environment:** Google Colab, Visual Studio Code
- **Version control:** GitLab

Chapter 2

Plan, Tasks and Deliverables

The project is structured into four phases:

- **Phase 1 – Literature review and data preparation (1 month)**
  Deliverables: Preprocessed dataset + CSV
- **Phase 2 – Dataset creation (1 month)**
  Deliverables: Prepared dataset for annotation
- **Phase 3 – Dataset annotation (2 months)**
  Deliverables: Annotated dataset
- **Phase 4 – Model fine tuning (1 month)**
  Deliverables: Fine-tuned model with metrics on a validation dataset

Chapter 3

Dataset review and normalisation

3.1 Dataset overview

The dataset used in this project was provided by medical experts from the hospital (CHU Orléans). It consists of folders containing microscopic images of urinary crystals. The folder names and image filenames include different codes and abbreviations that describe the crystal type and acquisition conditions.

To better understand and use this data we first analyze how the images are organized. An example of one image and its filename is shown below. Based on this structure we standardize the dataset to make it consistent and easier to use for fine tuning the model with our custom dataset.

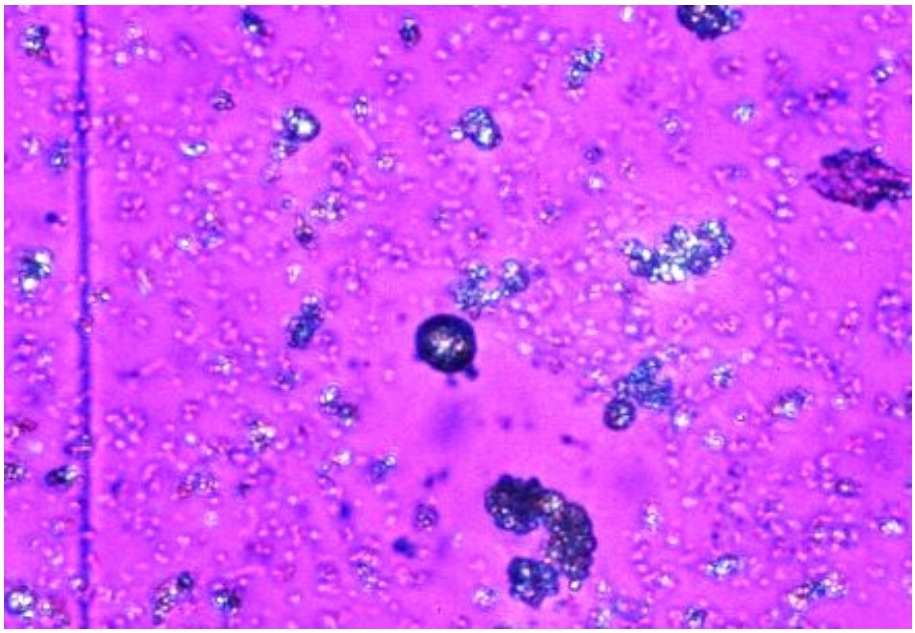


Figure 3.1: `AUampolaris3_pH5_9.jpg`

3.2 Coding guide: crystal species codes

To interpret folder names and filenames, a coding guide is used. It defines crystal species codes. The full list used in this project is shown below.

Table 3.1: Crystal species codes used in the dataset.

Code	Crystal species (as defined in the guide)
ACV	aciclovir
AMO	amoxicilline trihydratée
AMP	ampicilline trihydratée
ATZ	atazanavir
AU0	acide urique anhydre
AU1	acide urique monohydraté
AU2	acide urique dihydraté
AUam	acide urique amorphe
Br	brushite
C1	whewellite (oxalate de calcium monohydraté)
C2	weddellite (oxalate de calcium dihydraté)
C3	caoxite (oxalate de calcium trihydraté)
CAL	calcite (carbonate de calcium anhydre)
CEF	ceftriaxonate de calcium
CHOL	cholestérol
CIP	ciprofloxacine (sel de magnésium)
CRX	Crixivan ® = indinavir monohydraté
CYS	cystine
DHAd	dihydroxyadénine (dihydroxy-2,8-adénine)
LEU	leucine
MES	mésalazine
NASD	N-acétylsulfadiazine
NASM	chlorhydrate de N-acétylsulfaméthoxazole
OCP	phosphate octocalcique pentahydraté
PACC	phosphate amorphe de calcium carbonaté
PAM	struvite (phosphate ammoniacomagnésien hexahydraté)
SD	sulfadiazine
TYR	tyrosine
UrAm	Urate NH4 = Urate acide d'ammonium anhydre
UrNa	Urate acide de sodium monohydrate
UrNK	Urate NK = Urate double de sodium et potassium
VAT	vatérite (carbonate de calcium anhydre)
XAN	xanthine

3.3 Coding guide: abbreviations found in filenames

Filenames may contain abbreviations describing acquisition conditions, morphology, or context. The guide provides the following list of abbreviations.

Table 3.2: Abbreviations that may appear in image filenames.

Abbreviation	Meaning (as defined in the guide)
4C	urine gardée à +4°C
agr	agrégat(s)
aiguilles	cristaux en forme d'aiguilles
cacahuetes	cristaux (petits) en forme de cacahuètes
cuiller	cristaux en forme de cuiller chinoise
bil	bile

Abbreviation	Meaning (as defined in the guide)
bis	images prises en double avec la caméra fixée sur le microscope
corr	image corrigée (souvent luminosité, parfois cadrage)
croix	cristal en forme de croix ou en lames de ciseaux
CU	culot urinaire
cylind	cylindre tubulaire
det	portion agrandie d'une image plus large
diabolo	cristaux dimérisés en forme de diabolo
écailles	cristaux en forme d'écailles de poisson
EG	éthylène-glycol (intoxication à)
enf	enfant
enfh	garçon
enff	fille
fleur	cristaux en forme de fleur
helice	cristal en forme d'hélice
hexag	cristaux de forme hexagonale
HOP	hyperoxalurie primaire
lb	lumière blanche (sans polarisation)
macle	développement de nouveaux cristaux sur un cristal préexistant
navette	cristaux en navette
NB	image en noir et blanc
nez	kyste nasal
nuclheter	nucléation hétérogène
oxalose	diagnostic médical d'une hyperoxalurie primaire
pH	valeur du pH de l'urine
plum	plumeux
Pol	image en lumière polarisée
Polnoir	image en lumière polarisée sur fond noir
PV	plaque vitreuse
queue	cristaux présentant une excroissance latérale unique
spher	cristaux arrondis en forme de sphères
stade	cristal en forme de stade d'athlétisme
tab	cristaux tabulaires

3.4 Dataset issues identified during Phase 1

During the dataset review, several issues were identified. These issues explain why the folder structure cannot be used directly as ground truth.

3.4.1 Folder organization and label reliability

Images are organized by class in folders. Each folder indicates a class but there are exceptions. A crystal species can be pure or mixed with one or more other species.

The main issues are:

- In the same folder, images can contain crystals from different classes.
- A common situation is a folder named with one species, while the image contains multiple species.
- Sometimes the mixture is written in the filename but it is difficult to process automatically.

- Some images can be placed in the wrong folder (a rare case was reported).
- Some classes are missing because there are no images or too few images for rare species.

3.4.2 Morphological variability inside a species

A single crystal species can show a diversity of morphologies. This morphological diversity contributes to the identification difficulty.

3.4.3 Filename annotations are not standardized

Some filenames contain useful information. They may indicate clinical context, mixed classes, or acquisition information such as polarized light and pH.

However, automatic interpretation is difficult. Abbreviations are used and there is no strict naming standard. A suggested improvement is to normalize these textual indications and provide a clear guide.

3.4.4 Image Abbreviations

The dataset contains heterogeneous abbreviations:

- variable number of images per class,
- very different image resolutions,
- images with and without polarized light,
- different magnification factors without metadata,
- cropped images extracted from other images,
- duplicates or the same sample captured under different lighting conditions.

3.5 Metadata CSV to normalize information

To reduce ambiguity and centralize information, a metadata CSV file was created. The idea is to extract structured information from folder names and filenames.

3.5.1 How the CSV is built

The script uses two mapping files:

- a species mapping (variants → code and canonical name),
- an abbreviation mapping (variants → code and explanation).

For each image, the script:

- detects species in the folder name and in the filename,
- applies a fallback rule for filenames that are only registration numbers (species taken from the directory),
- sets a `mixture` flag when more than one species code is detected,
- extracts abbreviations from the filename using known variants,
- extracts pH values from patterns such as `pH85` or `pH8.5` and normalizes the format,
- stores a short note explaining how the species information was obtained.

3.5.2 CSV fields

The CSV is designed to keep both the raw file information and the normalized metadata in one place.

- `filename`: the original image filename. It often contains useful tokens such as species codes, abbreviations, and sometimes pH.
- `rel_path`: the relative path from the project root. This preserves the folder structure, since images are organized by folders that usually indicate a species.
- `species`: the detected species written as `CODE (canonical name)`. The species can be detected from the folder name and/or from the filename.
- `mixture`: `Yes` if more than one species code is detected, otherwise `No`.
- `abbreviations`: detected abbreviations written as `CODE (explanation)`.
- `ph`: extracted pH value when the filename contains a pH pattern (normalized format).
- `notes`: short information explaining how the species was obtained (for example, fallback to the directory when the filename is only a registration number).

In practice, the folder structure is used as a weak label signal. We first look for species codes in the filename. If none are found and the filename looks like a registration number, we take the species from the folder. We also detect whether the filename contains multiple species codes, which indicates a mixture.

3.5.3 Examples

The examples below show how filenames and folders are normalised into structured metadata. The values are shown exactly as they appear in the generated CSV.

- **Mixture + clinical context**
  `filename`: `C1 (+C2)_intoxication Ethylene_glycol (EG)_3.tif`
  `rel_path`: `RawData/SÇlection cristaux Hennet/whewellite/C1 (+C2)_intoxication Ethylene_glycol (EG)_3.tif`
  `species`: `C1 (whewellite (oxalate de calcium monohydraté)), C2 (weddellite (oxalate de calcium dihydraté))`
  `mixture`: `Yes`
  `abbreviations`: `EG (éthylène-glycol (intoxication à))`

- **Registration number (species from directory)**
  `filename`: `P9130977.JPG`
  `rel_path`: `RawData/SÇlection cristaux Hennet/whewellite/P9130977.JPG`
  `species`: `C1 (whewellite (oxalate de calcium monohydraté))`
  `mixture`: `No`
  `notes`: `species from directory (registration number)`

- **pH normalization from filename**
  `filename`: `calcite_pH85_C8097.jpg`
  `rel_path`: `RawData/SÇlection cristaux Hennet/calcite/calcite_pH85_C8097.jpg`
  `species`: `CAL (calcite (carbonate de calcium anhydre))`
  `mixture`: `No`
  `ph`: `8.5`

- **Polarized light indicator in filename**

filename: Calcite_pol_x200.jpg
rel_path: RawData/SÇlection cristaux Hennet/calcite/Calcite_pol_x200.jpg
species: CAL (calcite (carbonate de calcium anhydre))
mixture: No
abbreviations: pol (image en lumière polarisée)

- **No species detected**
  filename: P5092126.JPG
  rel_path: RawData/SÇlection cristaux Hennet/artefact/P5092126.JPG
  notes: species not found

Chapter 4

Dataset analysis

4.1 Metadata CSV analysis

The quantitative analysis performed on the metadata CSV is shown below. The analysis separates images into non-mixture cases (single species detected) and mixture cases (multiple species detected) and it also checks the presence of pH information.

4.1.1 Global split: non-mixture vs mixture

The dataset contains **659** images in total. Among them, **581** images are labeled as **non-mixture** and **78** images are labeled as **mixture**.

This confirms that the dataset contains both clean single-species images and real-world mixed samples. Non-mixture images are simpler to annotate and can be used as a starting point. Mixture images require more careful instance-level annotation.

4.1.2 Non-mixture images per crystal code

For non-mixture images, the number of images per species code is summarized in Table 4.1.

Species Code	Number of Images
C2	89
AMO	78
CYS	61
C1	55
AU2	42
Br	36
CAL	24
PAM	23
UrAm	15
DHAd	14
C3	11
NASD	11
CHOL	9
ATZ	7
CRX	5
SD	4
AU0	3
NASM	3
UrNK	2

Table 4.1: Distribution of non-mixture images per crystal species code.

This distribution shows strong imbalance across classes. Some species are well represented, while others have very few samples.

4.1.3 Mixture images per crystal code

For mixture images, the number of occurrences per species code is summarized in Table 4.2.

Species Code	Number of Images
AMO	38
C2	38
Br	25
PAM	18
UrAm	16
DHAd	11
CYS	10
C1	9
C3	6
AU2	2
AU0	1
NASD	1
SD	1

Table 4.2: Distribution of species within mixture images.

Only **65** images contain a pH value, while **594** images do not include pH information. This means pH cannot be used as a consistent feature across the dataset.

Chapter 5

Dataset Annotation

In this phase, the dataset is manually annotated to create reliable ground truth data. The annotation was performed using CVAT AI.

The annotation process was carried out by medical engineers from the hospital in Orléans, who have domain expertise in urinary crystal identification. Their role was to correctly identify and label crystal types in each image.

After the annotation was completed, the dataset was exported into a format suitable for training.

5.1 Chosen annotation format

Several export formats are available in CVAT. For this project, the dataset was exported using the **Ultralytics YOLO Segmentation 1.0** format.

This format was selected because the next step of the project is to fine-tune a YOLO-based model. Using a compatible format simplifies the training pipeline and avoids additional conversion steps.

YOLO segmentation is also well adapted to this problem, since crystals can have irregular shapes and multiple objects can appear in the same image. Polygon-based segmentation allows precise localization of each crystal instance.

5.2 Dataset structure

The exported dataset follows a standard structure required by Ultralytics YOLO:

- `images/` – contains all input images
- `labels/` – contains annotation files for each image
- `data.yaml` – configuration file describing the dataset

5.2.1 Images

The `images` directory contains the original microscopic images. The dataset is further divided into subset `train/` and each image keeps its original filename.

5.2.2 Labels (segmentation annotations)

For each image, there is a corresponding `.txt` file in the `labels` directory.

Each line in a label file represents one annotated object (one crystal instance). The format of each line is:

```
class_id x1 y1 x2 y2 x3 y3 ...  xn yn
```

where:

- `class_id` is the integer index of the crystal class,
- `x, y` are normalized coordinates (values between 0 and 1),
- the sequence of points defines a polygon around the crystal.

This representation allows precise segmentation of crystal shapes.

5.2.3 data.yaml configuration

The `data.yaml` file defines the dataset configuration used during training.

It includes:

- paths to the training and validation sets,
- the number of classes,
- the list of class names.

This file is required by the Ultralytics YOLO framework to correctly load the dataset and map class indices to labels.

5.3 Why YOLO segmentation format

The Ultralytics YOLO segmentation format was chosen for the following reasons:

- It is directly compatible with YOLO models, which will be used for fine-tuning.
- It supports multiple objects per image, which is essential for mixture cases.
- It provides polygon-based annotations, which are more precise than bounding boxes for irregular crystal shapes.
- It is lightweight and easy to use within a standard training pipeline in Google Colab.

Chapter 6

Model Fine Tuning (YOLO)

Phase 4 focuses on fine-tuning a YOLO segmentation model on the annotated dataset. The objective is to build a strong baseline and then improve it with a cleaner dataset and a better training setup.

6.1 Dataset split: labeled vs unlabeled

The full dataset contains **659** images. After the annotation phase (CVAT AI export), the dataset is divided into:

- **Labeled images: 612** (images that have annotation files)
- **Unlabeled images: 47** (images without usable annotations)

Only the labeled subset is used for training and validation. The unlabeled images are kept aside and are not used furthermore.

6.2 YOLO segmentation dataset format (training input)

The exported dataset follows the Ultralytics YOLO Segmentation format:

- `images/` contains the microscopic images
- `labels/` contains one `.txt` annotation file per image
- `data.yaml` contains dataset paths and the list of class names

Each label file stores one instance per line as:

```
class_id x1 y1 x2 y2 ...  xn yn
```

where the polygon points `(x, y)` are normalized coordinates.

This format is directly compatible with Ultralytics training and avoids any conversion step before fine-tuning.

6.3 Dataset preparation and training protocol

This section describes the full preparation pipeline used before fine-tuning, the strategy used to build reliable train/validation splits, and the two training configurations (baseline and im-

proved). The goal is to reduce noise, handle class imbalance, and ensure that evaluation is stable and comparable across runs.

6.3.1 Cleaning and preparation before training

Before training, the labeled dataset is cleaned to ensure that each sample is valid and usable for instance segmentation. Concretely, the cleaning step enforces the following rules:

- every training image must have a corresponding label file,
- empty or unusable annotations are removed,
- class indices remain consistent with the final `data.yaml` mapping.

After this step, the training source is a single clean dataset directory. This reduces label noise and prevents training from being affected by invalid samples.

6.3.2 Class filtering: keeping only classes with enough data

The dataset is strongly imbalanced. To avoid extreme sparsity, a simple filtering rule is applied: a class is kept only if it appears in at least **5 labeled images**.

After applying this threshold, the filtered training dataset contains **19 classes**. Table 6.1 summarizes the class distribution in the labeled subset used for training.

Class name	**Images**	**Instances**
AMO_amoxicilline_trihydratée	32	1357
ATZ_atazanavir	6	12
AU2_acide_urique_dihydraté	102	907
Br_brushite	62	955
C1_whewellite	70	461
C2_weddellite	135	485
C3_caoxite	25	29
CAL_calcite	23	980
CEF_ceftriaxonate_de_calcium	7	51
CHOL_cholestérol	9	47
CYS_cystine	97	644
Cylindre	7	11
DHAd_dihydroxyadénine	36	580
NASD_N-acétylsulfadiazine	15	80
NASM_N-acétylsulfaméthoxazole_chlorhydrate	6	141
PACC_Plaque_vitreuse	16	43
PAM_struvite	49	359
UrAm_urate_ammonium_anhydre	8	268
VAT_vatérite	15	82

Table 6.1: Filtered labeled dataset used for training (classes with at least 5 images).

This filtering step reduces extreme sparsity and makes both training and evaluation more realistic. It also prevents the validation signal from being dominated by classes with only a few samples.

6.3.3 Improved dataset variant: removing very small classes (drop4)

For an improved training setup, an additional manual filtering step is introduced to reduce instability caused by classes with very few labeled samples. Four classes are removed from the training label space:

Dropped class name
ATZ_atazanavir
Cylindre
CEF_ceftriaxonate_de_calcium
CHOL_cholestérol

Table 6.2: Classes removed in the `drop4` dataset to reduce sparsity and training instability.

After this step, the learning problem focuses on a smaller set of more consistently represented classes, which typically improves gradient stability and reduces noisy updates.

6.3.4 Train/validation strategy: 5-fold stratified K-Fold

A single random train/validation split is risky for this dataset because it is imbalanced and because some images contain multiple classes (mixtures). For this reason, a **5-fold stratified K-Fold** protocol is used.

The split is treated as a **multi-label** problem: for each image, a binary vector is built to represent which classes are present in its label file. Folds are then generated using a multilabel stratification method to keep class presence as balanced as possible across folds.

For each fold:

- one subset is used as validation,
- the remaining subsets are used for training,
- a dedicated `data.yaml` is generated to point to the corresponding train/val lists.

K-Fold statistics (filtered 19-class dataset)

After cleaning and class filtering, the number of clean usable images used for K-Fold is:

- **Clean usable images:** 607
- **Multi-label matrix shape:** `Y` has shape `(607, 19)`

The resulting fold sizes are:

Fold	Train	Val	Fold config
1	489	118	`data_fold_1.yaml`
2	483	124	`data_fold_2.yaml`
3	487	120	`data_fold_3.yaml`
4	481	126	`data_fold_4.yaml`
5	488	119	`data_fold_5.yaml`

Table 6.3: 5-fold stratified splits for the cleaned and filtered 19-class dataset.

K-Fold statistics (drop4 dataset)

After removing the four smallest classes, the same 5-fold protocol is applied to the `drop4` dataset. The resulting fold sizes are:

Fold	Train	Val	Fold config
1	469	114	`data_fold_1_abs.yaml`
2	463	120	`data_fold_2_abs.yaml`
3	466	117	`data_fold_3_abs.yaml`
4	467	116	`data_fold_4_abs.yaml`
5	467	116	`data_fold_5_abs.yaml`

Table 6.4: 5-fold stratified splits for the `drop4` dataset.

6.3.5 Training configurations

Two configurations are evaluated. The baseline validates the pipeline end-to-end with a small model and conservative settings, while the improved setup increases model capacity and uses controlled augmentation.

Baseline configuration

The baseline experiment is designed to validate the complete training pipeline with a lightweight YOLO segmentation model:

- model: `yolo11n-seg.pt`
- image size: `imgsz = 1024`
- epochs: `epochs = 50`
- batch size: `batch = 4`
- early stopping: `patience = 10`
- augmentations: disabled

Improved configuration

The improved setup is designed to increase model capacity and improve generalization on microscopic images:

- dataset: `dataset_clean_drop4`
- model: `yolo11m-seg.pt`
- image size: `imgsz = 1024`
- epochs: `epochs = 100`
- batch size: `batch = 4`
- early stopping: `patience = 20`
- augmentations: enabled and controlled:
  - `degrees = 15`, `translate = 0.10`, `scale = 0.50`
  - `fliplr = 0.5`, `flipud = 0.5`
  - `hsv_h = 0.01`, `hsv_s = 0.10`, `hsv_v = 0.10`
  - `mosaic = 0.0`, `mixup = 0.0` (kept disabled)

This configuration focuses on more stable learning by removing the sparsest classes and by using a larger model with custom augmentations.

6.3.6 Evaluation metrics (segmentation masks)

All reported metrics are taken from the segmentation evaluation for instance masks ("Masks"). The key metrics are:

- **Precision$_{\mathbf{M}}$**: fraction of predicted masks that are correct (lower false positives),
- **Recall$_{\mathbf{M}}$**: fraction of ground-truth masks that are detected (lower false negatives),
- **mAP50$_{\mathbf{M}}$**: mean Average Precision for masks at IoU threshold 0.50,
- **mAP50–95$_{\mathbf{M}}$**: mean Average Precision averaged over IoU thresholds 0.50–0.95 (step 0.05).

Because mAP50–95 is stricter and captures localization quality across multiple IoU thresholds, it is used as the main selection metric.

Best fold selection

For each configuration, five independent folds are trained. The best run is defined as:

Best fold = the fold with the highest mAP50–95$_{\mathbf{M}}$ on the validation set.

6.3.7 Baseline results (YOLO11n-seg, 19 classes)

The baseline experiment is evaluated using the best-fold criterion (highest mAP50–95$_{\mathrm{M}}$). The best baseline run is:

- **Best fold:** `fold_2`
- **Precision$_{\mathbf{M}}$:** 0.67119
- **Recall$_{\mathbf{M}}$:** 0.67146
- **mAP50$_{\mathbf{M}}$:** 0.66148
- **mAP50–95$_{\mathbf{M}}$:** 0.46970

Baseline confusion matrices

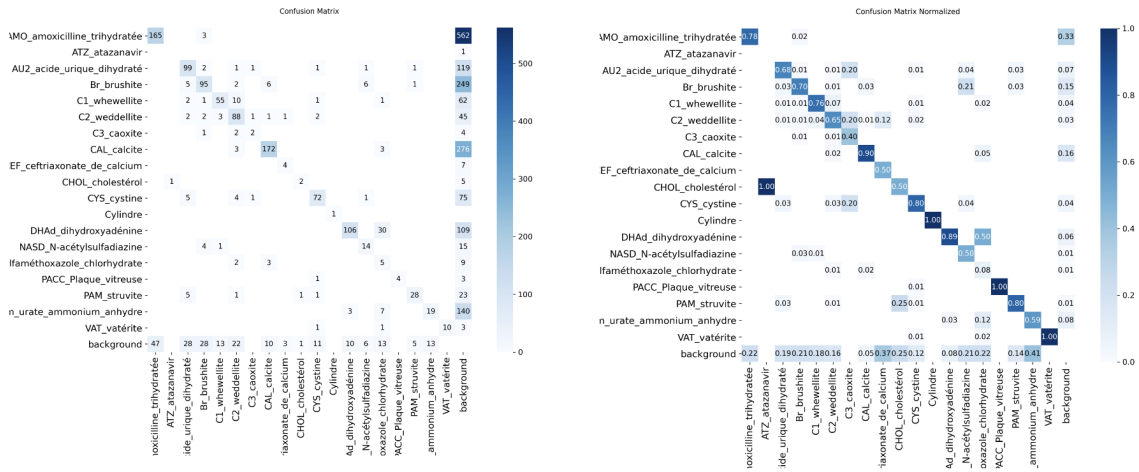


Figure 6.1: Baseline confusion matrices (raw counts on the left, normalized on the right).

From the normalized confusion matrix (Figure 6.1, right), several classes show a clear diagonal (good per-class correctness). For example, the diagonal values visible on the plot include:

- `CAL_calcite`: 0.90,
- `CYS_cystine`: 0.80,
- `DHAd_dihydroxyadénine`: 0.89,
- `AMO_amoxicilline_trihydratée`: 0.78,
- `AU2_acide_urique_dihydraté`: 0.68,
- `Br_brushite`: 0.70,
- `C2_weddellite`: 0.65,
- `C3_caoxite`: 0.40.

The raw confusion matrix (Figure 6.1, left) also shows a strong **background column** with large counts (e.g., 562 for `AMO_...`, 276 for `CAL_calcite`, 249 for `Br_brushite`, 140 for `UrAm_...`). This indicates that a substantial fraction of ground-truth objects are missed and end up predicted as background (false negatives), which is consistent with the baseline model capacity and conservative training setup.

Note: the normalized plot displays a negative value on the "background" row (e.g., -0.22). This is not meaningful for a standard normalized confusion matrix and is treated as a visualization/normalization artifact rather than an interpretable metric.

6.3.8 Improved results (YOLO11m-seg, drop4)

For the improved setup, results are summarized as mean ± standard deviation across folds (Masks):

- **Precision$_{\mathbf{M}}$:** 0.7022 ± 0.0335
- **Recall$_{\mathbf{M}}$:** 0.6306 ± 0.0513
- **mAP50$_{\mathbf{M}}$:** 0.6813 ± 0.0427
- **mAP50–95$_{\mathbf{M}}$:** 0.4874 ± 0.0453

The best improved fold (highest mAP50–95$_{\mathrm{M}}$) is:

- **Best fold:** `fold_1`
- **Best epoch:** 74
- **Precision$_{\mathbf{M}}$:** 0.70804
- **Recall$_{\mathbf{M}}$:** 0.70550
- **mAP50$_{\mathbf{M}}$:** 0.74196
- **mAP50–95$_{\mathbf{M}}$:** 0.54213

Improved confusion matrices

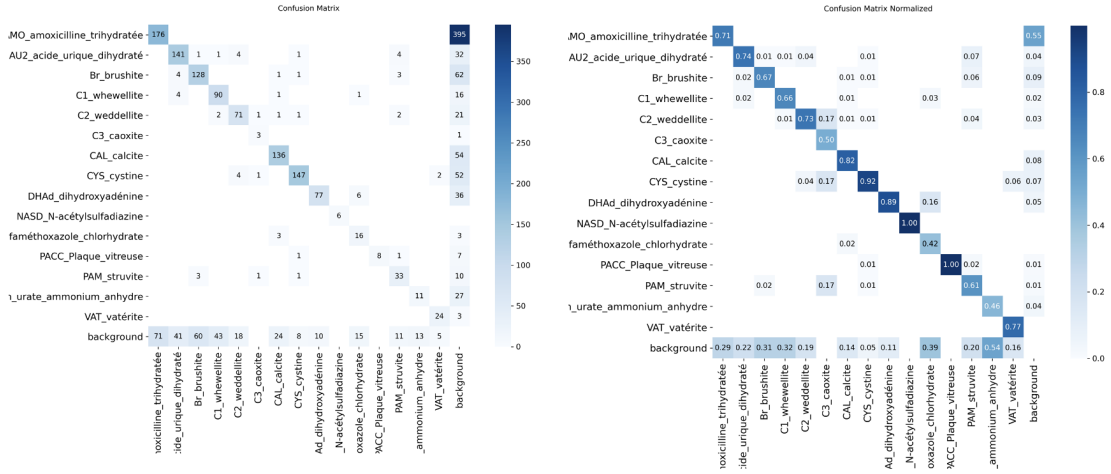


Figure 6.2: Improved confusion matrices (raw counts on the left, normalized on the right).

The normalized confusion matrix (Figure 6.2, right) shows a clean diagonal for several dominant classes. The diagonal values visible on the plot include:

- `CYS_cystine`: 0.92,
- `DHAd_dihydroxyadénine`: 0.89,
- `NASD_N-acétylsulfadiazine`: 1.00,
- `PACC_Plaque_vitreuse`: 1.00,
- `CAL_calcite`: 0.82,
- `AU2_acide_urique_dihydraté`: 0.74,
- `C2_weddellite`: 0.73,
- `AMO_amoxicilline_trihydratée`: 0.71,
- `Br_brushite`: 0.67,
- `C1_whewellite`: 0.66,
- `PAM_struvite`: 0.61,
- `UrAm_urate_ammonium_anhydre`: 0.46,
- `faméthoxazole_chlorhydrate`: 0.42,
- `VAT_vatérite`: 0.77,
- `C3_caoxite`: 0.50.

The raw matrix (Figure 6.2, left) still contains a large background column (e.g., 395 for `AMO_...`, 62 for `Br_brushite`, 54 for `CAL_calcite`, 52 for `CYS_cystine`). This shows that missed detections remain a major error mode. However, the improved best fold also reaches higher mAP50–95_{M} (0.54213), which indicates better mask localization and overall segmentation quality under the stricter IoU averaging.

6.3.9 Summary comparison (baseline vs improved)

Overall, the improved setup yields stronger strict segmentation performance:

- baseline best fold mAP50–$95_{\mathrm{M}} = 0.46970$,
- improved best fold mAP50–$95_{\mathrm{M}} = 0.54213$.

6.3.10 Validation batch visualization

During validation, the model is evaluated on the validation subset of the current fold. The validation images are not used for training updates. They are used only to measure model performance and generate predictions that can be compared with the ground truth annotations.

For each validation image, the model produces predicted instance masks together with class labels and confidence scores. These predictions are then compared with the ground truth masks provided in the label files.

Figure 6.3 shows an example of a validation batch visualization generated during evaluation. Each bounding box and mask corresponds to a predicted object together with the predicted class name and its confidence score.



Figure 6.3: Example predictions on a validation batch. Predicted masks, class labels, and confidence scores are displayed on validation images.

The visualization also shows that some images contain multiple objects of the same class. In these cases the model produces several instance masks each corresponding to an individual crystal structure. The predictions generally align with visible crystal shapes, while the confidence scores reflect the model's certainty for each detected instance.

Chapter 7

Conclusion

This project allowed me to work on a real interdisciplinary problem between computer science and medicine. The objective was to explore whether deep learning models can help recognize urinary crystals in microscopic images and potentially support experts in their analysis.

During the project, several steps were necessary before training a model. The dataset first had to be reviewed and normalized, because the original structure contained mixtures of species, inconsistent filenames, and heterogeneous image conditions. To better organize the information, a metadata CSV was created to standardize species codes, abbreviations, and other useful information extracted from filenames.

The images were then manually annotated by medical engineers using CVAT and exported in the Ultralytics YOLO segmentation format. After cleaning the dataset and filtering classes with too few samples, a training pipeline based on stratified 5-fold cross-validation was implemented.

A baseline YOLO segmentation model was first trained to validate the full pipeline. Afterwards, an improved configuration using a larger model and a cleaner dataset was tested. This setup achieved better segmentation results and more stable behavior across classes.

Overall, the project demonstrates that deep learning can be applied to this type of medical microscopic data, but it also highlights how important careful dataset preparation and annotation are for obtaining reliable models.